

Activity 10: Capstone: BudgetBuddy Release Evidence

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: A1, A2, A3, A4, A5

Estimated time: 105 minutes

Objective

Integrate the portable C++ core, Objective-C++ adapter and SwiftUI shell into a verified release evidence pack.

Deliverable

Working app, complete tests, architecture pack and deployment checklist

Tools

C++20, CMake, Xcode, SwiftUI, XCTest, documentation templates

Included files

- ``starter/capstone-brief.txt``
- ``starter/evidence-index.csv``
- ``starter/release-checklist.txt``
- ``solution/evidence-index.csv``
- ``solution/deployment-position.txt``
- ``project.yml`` — XcodeGen source of truth
- ``scripts/generate-project.sh`` — regenerates the shared Xcode project
- ``scripts/build-simulator.sh`` — resolves an installed iPhone simulator and builds
- ``scripts/test.sh`` — runs the Swift Testing target and any UI test target
- ``solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png`` — opaque course app icon
- ``solution/PrivacyInfo.xcprivacy`` — reviewed privacy manifest baseline
- ``solution/projects/BudgetBuddy/``, ``FocusCards/``, ``HabitPulse/`` — three complete standalone XcodeGen apps

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/10-capstone-release-evidence``, read ``starter/capstone-brief.txt``, and copy the checklist plus evidence index into ``workspace/``.
2. Create ``workspace/app``, ``workspace/tests``, ``workspace/docs`` and ``workspace/evidence``. Copy only the accepted outputs from Activities 1–9; exclude build caches, credentials, signing files and personal data.
3. Reconfirm the requirement baseline by comparing the Activity 1 matrix with the capstone brief. Record any change as an explicit decision before changing implementation.
4. Build and test the portable C++ core independently using ``cmake -S workspace/app/DomainCore -B workspace/app/DomainCore/build && cmake --build workspace/app/DomainCore/build && ctest --test-dir workspace/app/DomainCore/build --output-on-failure``, or the documented strict compiler fallback.
5. Build the Xcode scheme and run focused bridge tests. Record the scheme, destination and result; do not describe a Simulator build as an archive or App Store submission.
6. Run the SwiftUI golden journey: add an exact-cent expense, trigger and recover from an invalid entry, save, relaunch and verify stable identity and amount.
7. Run the accessibility critical path and persistence fault injection. Place raw results under ``workspace/evidence/`` and link them from the scorecard.
8. Reconcile C4 diagrams, ADRs and the public bridge contract against the final code. Resolve any dependency or ownership mismatch before marking documentation complete.
9. Complete ``workspace/evidence-index.csv`` with artifact version, requirement IDs, owner, evidence path and status. No PASS row may have an empty evidence field.
10. Review ``starter/release-checklist.txt``. Mark the archive item as planned unless a signed Release archive was actually created and validated; never infer App Store readiness from local tests.
11. Write ``workspace/docs/deployment-position.txt`` using the bounded pattern in the solution: distinguish local release-candidate evidence from signing, upload, processing and submission gates.
12. Run ``rg -n -i 'api[_]?key|secret|password|BEGIN .*PRIVATE KEY' workspace || true``. Investigate every match and remove real secrets before packaging.
13. Validate that every requirement has design, implementation, test and evidence links. Ask a reviewer to reproduce one domain check and one bridge/UI check from the index.
14. Create the final inventory with ``find workspace -type f | sort > workspace/evidence/file-inventory.txt``. Retain sources, fixtures, tests and evidence; remove only caches, derived data and temporary stores.
15. Regenerate and verify each standalone project under ``solution/projects/BudgetBuddy``, ``FocusCards`` and ``HabitPulse``. In each folder run ``./scripts/generate-project.sh``, ``./scripts/build-simulator.sh`` and ``./scripts/test.sh``; record the resolved simulator name and exact build/test result.
16. Launch each app on the same available iPhone Simulator. Confirm its title, primary interaction and accessibility identifiers, then capture one evidence screenshot after tests pass. Inspect the compiled icon on the Simulator home screen as well as the 1024px source; file presence alone is insufficient.

17. Create the App Store Connect preflight. Reconcile the app record identity with the bundle ID, marketing version and integer build number; review the opaque 1024px icon, privacy manifest, required-reason APIs, usage descriptions, entitlements, encryption declaration, device family and Release settings.
18. Prepare the signed-archive gate: specify the Release scheme and archive path, validate the archive in Xcode, prepare an export configuration from trainer-supplied signing values, export the IPA and validate it before upload. Mark this TRAINER DEMO / FUTURE CREDENTIALLED GATE unless valid signing access exists; never add credentials to the Activity.
19. Prepare the App Store Connect record: complete description, subtitle, keywords, support/privacy URLs, category, copyright and App Review contact; prepare accepted-size screenshots from real Simulator states and map each image to its required display size.
20. Review and publish App Privacy answers from the verified data-flow inventory; answer the age-rating and content-rights questions; set pricing and availability explicitly. Do not infer any answer from an AI-generated manifest or from price configuration alone.
21. After a credentialled trainer uploads, wait for the intended build to finish processing, reconcile version/build/bundle identity, attach only that build, resolve Apple-reported blockers, submit for review and read back the app name, version, build, availability and review state. Until that happens, keep every external row HOLD with owner and next action.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Integrate and verify the three supplied simulator-ready projects: BudgetBuddy, FocusCards, and HabitPulse. For each, regenerate from project.yml, build on a simulator resolved from xcrun simctl list devices available, run Swift Testing and XCTest UI tests, inspect the compiled 1024px AppIcon, and retain results. For BudgetBuddy also prove exact cents across the Objective-C++/C++ seam. Then generate a release-candidate evidence pack and an App Store Connect handoff covering identity/version/build, privacy manifest, required-reason APIs, archive, validation, export, upload, metadata, screenshots, App Privacy, age rating, content rights, pricing/availability, processed-build attachment, submission and Apple-side readback. Mark credentials, signing, upload and real submission as TRAINER DEMO / FUTURE CREDENTIALLED GATE; leave each unexecuted gate HOLD. Output exact paths and complete files with no secret values or submission claim.

Review and repair prompt

Audit all three projects and the release handoff for placeholders, partial files, target-membership drift, missing/alpha icons, contract drift, C++ leakage, duplicated policy, unsafe conversions, weak error mapping, @Observable misuse, wrong test framework, unstable UI automation, persistence loss, privacy/signing drift, stale metadata, missing availability, secrets, and unsupported release claims. Rank findings and repair the smallest causal layer. Rerun generation/build/tests, launch each app on the resolved Simulator, and require complete traceability. External gates remain HOLD until a trainer performs them with current Apple-side readback.

****Protected file scope:**** verify-local.sh; CI workflow; privacy-security-review.md; evidence-index.csv; release-scorecard.json; release-notes.md; deployment-position.md; file-inventory.txt; final-audit.md

****Expected verification:**** All three generated Xcode projects build, test and launch on a resolved iPhone Simulator; BudgetBuddy proves exact cents through Objective-C++; icons and privacy manifests are present; local evidence is complete; every unexecuted archive/upload/metadata/privacy/age/availability/attach/submit/readback gate remains HOLD.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

A clean evidence index proves requirements, implementation, interoperability, documentation and deployment readiness without claiming App Store submission.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

When a system test fails, localise the fault by rerunning domain then bridge tests before editing UI code.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Remove local build caches and secrets; retain source, fixtures, tests and evidence pack.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.